Phase III - AI Coding, Vibe Coding and Baseline Validation

Project: Comparative Analysis of AI-Assisted and Manual Data Labeling Accuracy Across Scientific Datasets **Phase:** III | **Log date:** 2026-10-28 | **Due:** 30 Oct 2026 **Implements:** `phase2_experiment_plan.md` (D-018 to D-023). Earlier stages: `gap_matrix.md`, `research_decisions.md`, `literature_to_experiment_bridge.md`, `feasibility_novelty_matrix.md`.

Tags: VERIFIED = produced by code that ran in this workspace, output saved in `results/`, or read in a file opened here. INFERRED = my reasoning, stated. UNKNOWN = not checked; the check is named. **Standing rules (D-003):** every number in this file comes from a saved output. Nothing from synthetic data is reported as a result.

---

0. Status in one table

Part	Status	Evidence
Study B code (routing, bootstrap, label loader, runner, model code)	Written	Section 2
Unit tests with known answers	**15 of 15 pass**	`results/test_log.txt`
Gate G-4: raw file reproduces `counts.npy`	**Pass** (exact)	`results/label_only.json`
Label-only baseline (manual arm, noise floor, reference sensitivity)	**Done on real CIFAR-10H data**	Section 4
Pipeline check on a real classifier (bundled digits)	Done; **not a Study B result**	Section 5
Model arm on CIFAR-10 images	**NOT RUN.** CIFAR-10 images and torch were unreachable in this workspace.	Section 7
`extract_features.py` (torch code)	**Written, never executed**	Section 2

The central result of Study B (does uncertainty routing beat random review with a real classifier) does not exist yet. It needs one run on your machine or a notebook (Section 7).

---

1. Environment and what was blocked

Item	Finding	Tag
Hardware here	2 CPU cores, 7 GB RAM, no GPU	VERIFIED
Python and libraries	Python 3.13.17, numpy 2.5.3, pandas 3.0.5, scikit-learn 1.9.1, scipy 1.18.1 (`results/env.txt`)	VERIFIED
CIFAR-10H data	Cloned from the public repository (commit 389f22d, 23 Mar 2026). Checksums in `results/data_sha256.txt`.	VERIFIED
CIFAR-10 images	Download refused by the workspace network policy	VERIFIED
torch, torchvision	`pip` found no matching distribution	VERIFIED
pytest	`pip` found no matching distribution; a 20-line runner (`run_tests.py`) replaces it	VERIFIED
Backbone weights and licence	Not opened. Candidate is the torchvision ResNet-18 ImageNet weights; terms UNKNOWN.	UNKNOWN

---

2. AI coding and vibe-coding log

How the code was produced. The code was generated by an AI assistant from the written protocol, not typed by hand. Because generated code can look right and be wrong, each piece was tied to a check that does not depend on trusting it.

File	Role	How it was checked
`routing.py`	Uncertainty scores (entropy, margin, least confidence); arm selection with seeded random tie-breaking; oracle arm; single-annotator draw	Hand-built probabilities with known orderings; size and uniqueness checks; tie-breaking test; closed-form oracle accuracy
`stats.py`	Paired bootstrap over items	Constructed pairs with a known difference, a zero difference, and an all-zero difference
`labels.py`	Rebuild per-item judgments from the raw file; reference labels	**Gate G-4** against the published `counts.npy`
`experiment.py`	All arms x budgets x reviewers x seeds; summary with paired intervals	Gates G-2, G-3, G-6; digits pipeline check
`model.py`	Hyperparameter choice on a held-out part of the train set; per-seed 80% training; calibration error	Digits pipeline check; never reads the test set
`run_label_only.py`	Label-only baseline	Rerun gives byte-identical output
`run_model_arm.py`	Model arm runner; stops if the feature labels differ from CIFAR-10H's labels (G-8)	Ran on **synthetic** features (code path only)
`extract_features.py`	Frozen ResNet-18 features on CIFAR-10	**Not executed anywhere.** Needs a first run by you.
`validate_digits.py`	End-to-end check on bundled digits	Ran; output in `results/validate_digits_log.txt`
`run_tests.py`, `tests/test_core.py`	Test runner; 15 tests	Ran

Defects found during checking (all recorded, none hidden)

#	What happened	Cause	Fix	Tag
1	`test_margin_order_known` failed on first run	The AI-written test data had two items with the same margin (0.01). The routing code was correct; the **test** was wrong.	Changed the test probabilities and checked entropy and margin values by hand (printed in the session)	VERIFIED
2	`test_select_top_uncertain` failed	The test expected items {2, 1}, but item 3 (entropy 0.802) is more uncertain than item 1 (0.74 in the original data). Test wrong, code correct.	Same data change; entropy now 0.112, 0.856, 1.099, 0.802	VERIFIED
3	pytest and torch not installable	Workspace policy	Own runner; model arm moved to your machine	VERIFIED

Both test failures were errors in the AI's expected values, found because the tests were run. A passing test suite is only as good as its expected values, which is why the two fixes were done by hand arithmetic and not by editing until green. The tests do **not** prove the code is bug-free; they cover the cases in the table.

Risks of vibe coding that remain (INFERRED)

1. Tests were written by the same assistant that wrote the code, so a shared misunderstanding of the protocol could pass every test. Your read-through below is the independent check.
2. The torch script has never been run.
3. Logistic regression is capped at 300 iterations; whether it converges on 512-dimensional features is UNKNOWN. sklearn prints a warning if not; read the output.

What you must read yourself before trusting any number (do not skip)

#	Where	What to confirm
R1	`routing.select_for_review`	Higher score means reviewed first; ties are random; oracle ranks the model's real errors first
R2	`experiment.run_experiment`	The single-annotator draw is made once per seed and shared by every arm and budget (this is what makes comparisons paired); the random arm uses its own seeded stream
R3	`final_labels`	Reviewed items take the reviewer's label; all others keep the pre-label
R4	`labels.build_judgments`	Attention-check trials and the 16 excluded annotators are removed; no other filtering
R5	`run_model_arm.py`	The test features and labels are never used in `choose_C` or `probs_by_seed`
R6	Plan vs code	Primary contrast is entropy vs random, single-annotator reviewer, b = 0.20 (D-021)

3. Validation gates

Gate	Check	Result	Evidence
G-1	Routing on hand-made probabilities	**Pass** (tests for entropy, margin, least confidence, selection, oracle, ties, draw)	`test_log.txt`
G-2	Perfect reviewer, oracle arm = 1 - max(E - k, 0)/N	**Pass** on synthetic data (test) and on the digits run for all four budgets (exact to 1e-12)	`test_log.txt`, `validate_digits_log.txt`
G-3	Perfect reviewer at b = 1.0 gives 1.0 for every arm	**Pass**	`test_log.txt`
G-4	Raw file rebuilds `counts.npy` exactly	**Pass** under the rule "drop attention-check trials and annotators below 75% on them"; **fails** with no exclusion (2,834 items differ)	`label_only.json`
G-5	Bootstrap covers a known difference and zero	**Pass**	`test_log.txt`
G-6	Same seed, same output	**Pass** (unit test; label-only run reproduced identically)	`test_log.txt`
G-7	End-to-end pipeline on a real classifier	**Pass** (runs; results consistent with G-2/G-3); not a Study B result	Section 5
G-8	Feature-file test labels equal CIFAR-10H's `true_label`	**Not yet run**: needs the CIFAR-10 images. The runner stops if it fails.	`run_model_arm.py`

4. Baseline results on real data (label-only)

These use only the real CIFAR-10H annotator file and the original CIFAR-10 test labels. No classifier is involved. Source: `results/label_only.json`, `per_annotator_accuracy.csv`.

4.1 Reconciliation with the published files (G-4)

Quantity	Value	Tag
Rows in the raw file	539,910	VERIFIED
Attention-check rows	25,710	VERIFIED
Real judgments with no exclusion	514,200 (539,910 - 25,710)	VERIFIED
Annotators below 75% on attention checks	16 of 2,571	VERIFIED
Judgments after excluding them	511,000, **identical** to `counts.npy` (sum 511,000; 47 to 63 per item)	VERIFIED

Discrepancy to flag: the dataset's documentation page (read in Stage 2) says 14 workers were removed and 511,400 classifications were collected. The released files give 16 removed and 511,000. 2,555 annotators x 200 items = 511,000. Why the

page and the files differ is UNKNOWN. My code follows the released files, because that rule reproduces `counts.npy` exactly. Do not quote the 14 or the 511,400 for this analysis.

4.2 Reference-label sensitivity (D-020)

Quantity	Value
Panel plurality equals the original CIFAR-10 label	99.21% of items (9,921 of 10,000)
Items where they disagree	79
Plurality ties (broken by lower class index)	3

So the two reference rules differ on under 1% of items (INFERRED: the sensitivity analysis will probably shift accuracies by well under one point; the model arm will show).

4.3 Manual arm (G1): one annotator's label per item

Reference	Mean accuracy over 1,000 random draws	SD	2.5th to 97.5th percentile
Original CIFAR-10 label (primary)	0.9523	0.0018	0.9488 to 0.9558
Human plurality (sensitivity)	0.9544	0.0019	0.9508 to 0.9582

A model has to reach about 0.95 to match a single crowd annotator on this set (INFERRED from the table).

4.4 Noise floor (G5): annotator-to-annotator spread

Quantity	Value
Annotators analysed	2,555 (each labelled exactly 200 items)
Accuracy against the original label: mean	0.9524
SD	0.0287
5th percentile, median, 95th percentile	0.905, 0.960, 0.985

Reading (INFERRED). With 200 items and accuracy near 0.95, pure sampling noise would give an SD of about 0.015 (sqrt(0.95 x 0.05 / 200)). The observed SD of 0.029 is about twice that, so annotators genuinely differ, beyond chance. Any model-versus-annotator gap smaller than about 0.03 would sit inside one SD of that spread (this is the comparison H4 will make once the model accuracy exists). I have not computed a formal test of this.

4.5 Where single-annotator errors sit

Items	Count (of 10,000)
Every judgment correct	4,392
Fewer than 80% of judgments correct	609
Fewer than 50% of judgments correct	122

Why it matters for H2 (INFERRED, not tested): a single annotator's errors are concentrated on about 6% of images. If the model's uncertainty points at those same images, sending them to a single annotator helps little; if it points at images humans find easy, review helps more. The model arm is the test.

---

5. Pipeline check on a real classifier (NOT a Study B result)

Data: scikit-learn's bundled digits (899 test items after a 50/50 split), logistic regression, 10 seeds. Reviewer judgments are **synthetic** (20 per item; each correct with probability 0.9, otherwise a random class). This only shows the code runs end to end and obeys the gates. Digits are not a scientific dataset and say nothing about CIFAR-10H.

Model-only accuracy 0.9608; calibration error 0.0933 (seed 0). At budget 0.20:

Reviewer	Arm	Accuracy	Gain over random (paired 95% interval)
perfect	random	0.9677	-
perfect	entropy	0.9966	0.0288 (0.0207 to 0.0385)
perfect	margin	0.9959	0.0281 (0.0201 to 0.0378)
perfect	oracle	1.0000	0.0323 (0.0234 to 0.0429)
single (synthetic)	random	0.9471	-
single (synthetic)	entropy	0.9771	0.0300 (0.0214 to 0.0399)
single (synthetic)	oracle	0.9791	0.0320 (0.0231 to 0.0409)

The ordering (oracle >= uncertainty arms > random) and the exact oracle formula match what the protocol requires. Source: `results/validate_digits.csv`.

Synthetic-feature smoke test of `run_model_arm.py`. The runner was executed on the real CIFAR-10H labels with synthetic features generated from the labels. It completed (3 seeds, 300 bootstrap resamples, about 8 seconds) and wrote both result tables. **No numbers from this run are reported**, because the features were built from the answers and mean nothing. Its timing does not

predict the real run.

6. Deviations and implementation choices not fixed in the plan

#	Item	Status
1	No change to hypotheses, arms, budgets, reference rule or primary contrast.	VERIFIED
2	Choices the plan left open (all PROPOSED, D-024): features standardised before logistic regression; C chosen from {0.01, 0.1, 1.0} on a held-out 20% of the train set; 300 iterations; 15 bins for calibration error; ties in the plurality label go to the lower class index.	PROPOSED; confirm
3	Reconciliation rule in G-4 discovered by trying two rules; the 75% rule was in the plan only as "attention-check and exclusion rules".	VERIFIED
4	The plan said the label-only analysis would include a per-annotator noise floor; the SD comparison to sampling noise in 4.4 is added and is descriptive.	VERIFIED

7. What remains: the model arm (run outside this workspace)

Commands (any machine or Colab; first line installs, then run in order). Paths are relative to a working folder with the code files.

```
pip install torch torchvision scikit-learn pandas scipy numpy
git clone https://github.com/jcpeterson/cifar-10h.git
mkdir -p data_c10h && cp cifar-10h/data/cifar10h-counts.npy data_c10h/ && unzip cifar-10h/data/cifar10h-raw.zip -d data_c10h/raw
python run_tests.py
python extract_features.py --limit 500 --out timing.npz        # timing test; send me timing.npz.meta.json
python extract_features.py --out feats.npz                     # full run
python run_model_arm.py --features feats.npz --c10h data_c10h --out results_model
```

What to send back: `feats.npz.meta.json`, `results_model/results_primary_original_label.csv`, `results_model/results_sensitivity_human_plurality.csv`, `results_model/model_summary.json`, and any warnings printed (especially a convergence warning).

Pass conditions on your run: `run_tests.py` shows 15 passed; `run_model_arm.py` does not stop at G-4 or G-8; the `perfect` / `oracle` rows at b = 0.40 are exactly consistent with the closed form (the runner does not print this check; I will compute it from the CSV when you send it).

Interpretation rules, fixed now: H2 is judged only on the primary row (single, entropy vs random, b = 0.20). If its interval includes zero or is negative, report that as the result. The other rows are descriptive.

Risk on your run	Fallback
Cannot get GPU; extraction too slow at size 224	`--size 64` (lower accuracy, state it) or a subset using `--limit` (state it)
torchvision weights unusable or licence not acceptable	Another openly licensed backbone after reading its licence; or a small CNN trained from scratch (more compute)
Logistic regression does not converge	Raise iterations in `model.py`, log the change as a deviation
You cannot run it before 30 Oct	Hand in plan, validated code, the label-only baseline (Section 4), the digits check (Section 5) and an explicit statement that the model arm is pending

8. Decision log additions

ID	Decision	Status	Next action
D-024	Accept the implementation choices in Section 6, item 2	PROPOSED	You confirm or change
D-025	Follow the released files, not the documentation page, for the number of removed annotators (16) and judgments (511,000)	PROPOSED	Optionally email the dataset authors; note in the paper
D-026	Model arm to be run by you; no model-arm number appears anywhere until it has been	PROPOSED	You run Section 7
D-027	Derived tables from CIFAR-10H (`label_only.json`, `per_annotator_accuracy.csv`) carry CC BY-NC-SA 4.0 and say what changed	VERIFIED rule (D-022)	Add to the paper's data statement

Still open from earlier stages: backbone licence, scientific second dataset (V10, V11), hardware and exact deadline (O-5, O-6), authorship of the source review (D-004), searches S1 to S5 (D-015).

Next actions: N1 read R1-R6 in Section 2; N2 confirm D-018 to D-027; N3 run Section 7 and send the files; N4 I compute the primary contrast and write Phase IV (results and manuscript) from your output only.

9. Files and reproducibility

File	Purpose
`study_b/` (`routing.py`, `stats.py`, `labels.py`, `experiment.py`, `model.py`, `report_tables.py`, `run_label_only.py`, `run_model_arm.py`, `extract_features.py`, `validate_digits.py`, `run_tests.py`, `tests/test_core.py`, `README.md`)	Code (zip)

`results/` (`test_log.txt`, `label_only.json`, `label_only_log.txt`, `per_annotator_accuracy.csv`, `validate_digits.csv`, `validate_digits_log.txt`, `env.txt`, `data_sha256.txt`)	Saved outputs

Raw CIFAR-10H data is not included in the zip (clone it from the source). Seeds and RNG streams are fixed in the code; the label-only run was repeated and produced identical output.

Bridge: original papers -> selected method -> executable baseline -> planned test -> novelty and feasibility gate -> pre-specified protocol -> validated code and label-only baseline (this file) -> model-arm run (yours) -> results and manuscript (Phase IV).